

What CPSS Does in Our CAS Pipeline

Simple explanation, with real clauses from our own run

0. The whole thing in one picture

Trained TM $\rightarrow$ **Basis Z** $\rightarrow$ **CPSS** $\rightarrow$ **Signature**
2,400 clauses $\rightarrow$ flows $\times$ clauses $\rightarrow$ vote over $2B$ halves $\rightarrow$ few stable clauses
(6 classes $\times$ 400) $\rightarrow$ 0/1 matrix $\rightarrow$ keep $\hat{\pi}_j \geq \pi_{\text{thr}}$ $\rightarrow$ S^+ and S^-

The idea in one line. A clause can look useful by luck on one split of the data. So we re-split the data many times, fit a sparse model every time, and keep only the clauses that get picked *again and again*. That is CPSS — Complementary-Pairs Stability Selection (Shah & Samworth, 2013).

1. Which basis am I taking? (What goes into CPSS)

This is the most important thing to get right: **CPSS does not look at the raw network features at all.** The basis — the columns of the matrix we give to CPSS — is the **clause activations**.

	Raw feature space	Our basis Z (what CPSS sees)
Columns	22 numeric flow features	2,400 clauses (6×400)
Values	real numbers	0 or 1 (clause fired / did not fire)
Meaning	“Rate = 41,233 pkt/s”	“rule #436 matched this flow”

So one row of Z = one network flow. One column of Z = one clause (one learned rule). The entry is $z_j(x) \in \{0, 1\}$.

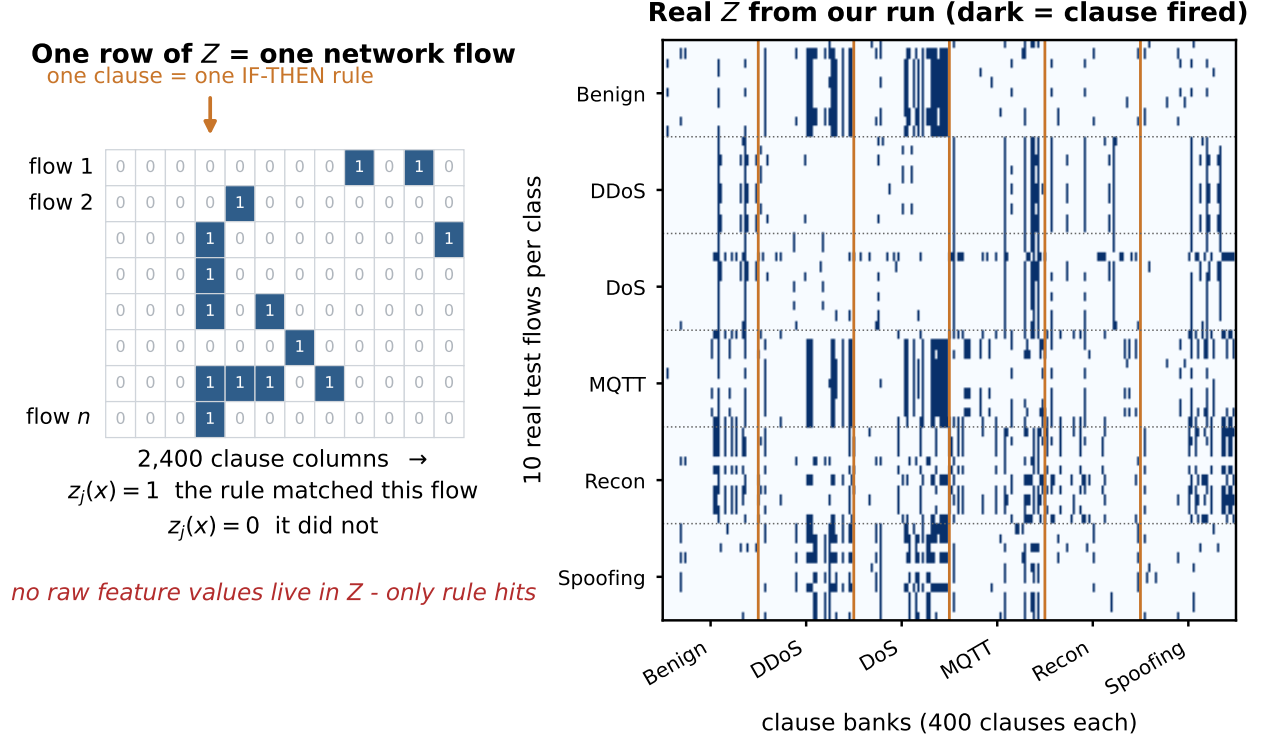


Figure 1: Left: the shape of Z . **Right:** the real matrix from our seed-42 run — 10 test flows per class (rows) against all 2,400 clauses (columns, split into the six banks by the orange lines). Each dark pixel is one clause firing on one flow. Flows of the same class light up recognisably similar column patterns: that pattern *is* the signature CPSS goes looking for.

1a. So what does a clause activation actually contain?

Nothing but a bit. $z_j(x) \in \{0, 1\}$, and that is the whole content: *did rule j match this flow, yes or no*. No confidence, no weight, no feature value. The rule’s weight $w_{c,j}$ and its reliability $\hat{\pi}_j$ live elsewhere and are applied later.

Concretely, the object we hand to CPSS in our run is:

Property	Measured value (seed 42, test set)
Shape	$80,868 \times 2,400$ (flows $\times$ clauses)
Storage type	<code>uint8</code> , ≈ 194 MB
Values present	exactly $\{0, 1\}$ — nothing else
Clauses firing per flow	median 283 of 2,400 (min 138, mean 306, max 708)
Overall density	12.8% ones
Per-clause firing rate	median 5.6%, 75th pct. 19.1%, 90th pct. 44.1%
Clauses that never fire	817 of 2,400 (on a 20k-flow sample)
Clauses that always fire	0

Two things follow from these numbers. First, Z is **sparse but not tiny**: about 300 rules match any given flow, so a flow is described by a *crowd* of overlapping rules, not one. Second, **a third of the pool is dead weight** — 817 clauses never fire at all, so they can never be selected. This is exactly the redundancy that clause pruning removes.

The columns have structure. Column j decodes as $\text{bank} = \lfloor j/400 \rfloor$ and, inside the bank, the

first 200 are *positive-polarity* clauses (trained to fire on their own class) and the last 200 are *negative-polarity* (trained to fire on everything *except* their own class). That structure is visible in the raw counts:

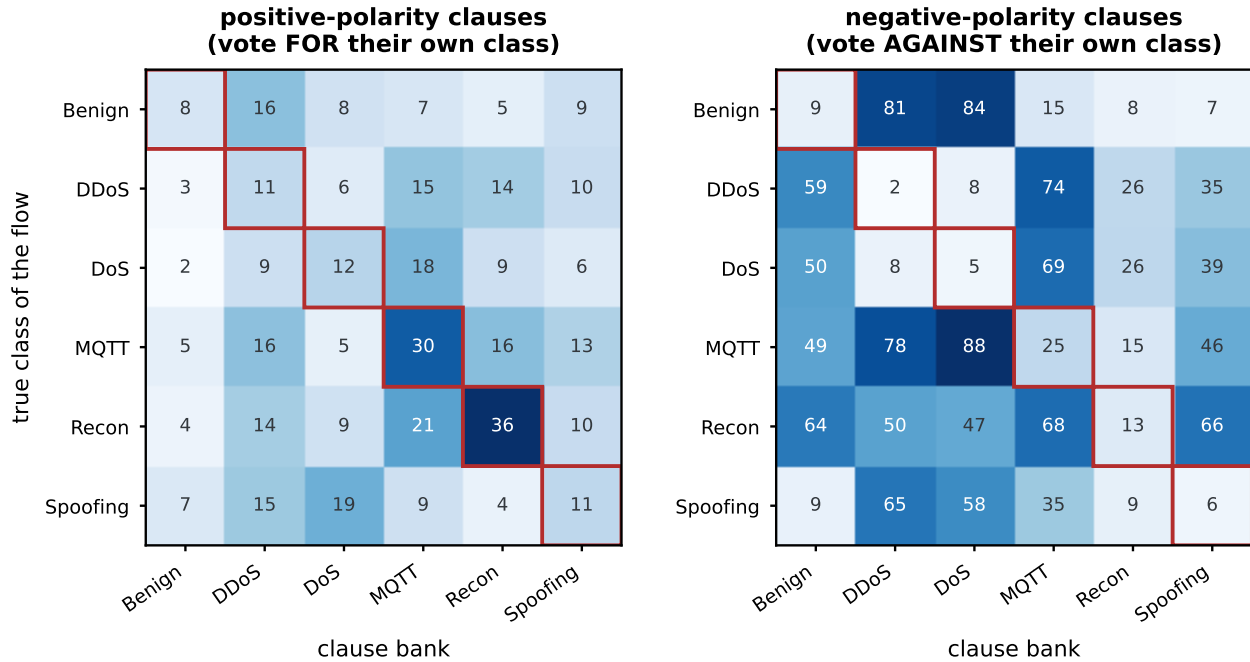


Figure 2: Mean number of clauses firing per flow, by the flow’s true class (rows) and the bank the clause came from (columns), out of 200 per half-bank. Red boxes mark the flow’s own class. The negative-polarity half is near-silent on its own class — Benign flows fire only 9 of Benign’s 200 negative clauses but 81–84 of DDoS’s and DoS’s — which is precisely what negative-polarity training is supposed to produce. Real data, 20,000 test flows.

Why this basis and not the raw features? Because we want the explanation to be made of *rules a human can read*, not of raw numbers. Every column we select is already an IF-THEN rule (Section 6 shows how we print it back out). Selecting features would give us “Rate matters”; selecting clauses gives us “*IF* Rate $\geq$ bin7 *AND NOT* Header.Length $\geq$ bin4 *THEN* DDoS”.

One subtle point. The pool is *all* 2,400 clauses, not only the 400 clauses of the class being explained. So the DDoS signature is allowed to borrow a clause out of the Benign bank if that clause is a reliable DDoS indicator. This really happens in our run (see Section 6).

2. What is B ? (How the data is split)

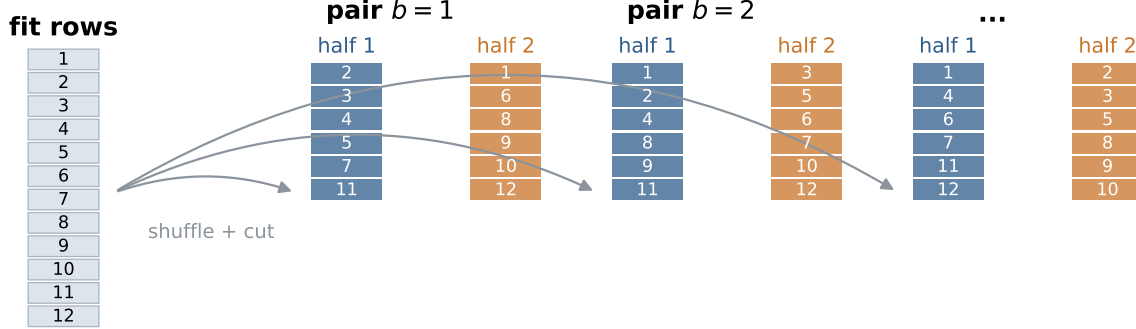
B is the **number of complementary pairs**.

One “complementary pair” = shuffle the fit rows, cut them into two *disjoint* halves. Half A and half B together cover the data exactly once, with no overlap — that is what “complementary” means.

Do that B times $\Rightarrow$ you get $2B$ halves in total.

Pair	Half 1	Half 2 (the complement)
$b = 1$	rows $\{3, 7, 8, \dots\}$	rows $\{1, 2, 4, \dots\}$
$b = 2$	(new shuffle) rows $\{1, 5, 9, \dots\}$	rows $\{2, 3, 6, \dots\}$
$b = 3$	(new shuffle) $\dots$	$\dots$

With $B = 15$ we fit $2B = 30$ half-models per class. A clause that survives all 30 different views of the data is genuinely stable; one that appeared only because of a lucky subset will not.



the two halves of a pair are **DISJOINT** and together cover the data exactly once $\Rightarrow$ 'complementary'

B pairs $\Rightarrow 2B$ halves ($B = 15 \Rightarrow 30$ fits per class)

Figure 3: One complementary pair = one shuffle, cut into two disjoint halves. Repeat B times to get $2B$ halves.

Why *complementary* halves and not just random subsamples? Because the Shah–Samworth error bound in the paper (Eq. 9) is proved for complementary pairs. If we used ordinary bootstrap resamples we would keep the intuition but lose the bound. That is the only reason for the pairing.

What changes if I change B : larger B = more re-splits = the stability score $\hat{\pi}_j$ is measured more finely (with $B = 5$ the score can only be $0, 0.1, 0.2, \dots, 1.0$; with $B = 25$ it has 50 possible levels) and is less noisy — but it costs $2B \times 6$ logistic fits per grid point. We sweep $B \in \{5, 10, 15, 20, 25\}$ and pick B on the *selection* partition, never on the report partition.

3. How is the fit done? (What happens inside one half)

On *each* of the $2B$ halves, for *each* class ℓ , we do this:

1. **Make a one-vs-rest label.** For DDoS: $y = 1$ if the flow is DDoS, else $y = 0$.
2. **Fit an ℓ_1 logistic regression** of y on the clause columns Z . Settings: `liblinear` solver, `class_weight="balanced"` (so rare classes are not drowned out), `max_iter=200`. The ℓ_1 penalty pushes most of the 2,400 coefficients to *exactly* zero — so the fit itself is a selector: the clauses left with a non-zero coefficient are the ones it chose.
3. **Repeat over 6 regularisation strengths** $\kappa \in \text{geomspace}(0.001, 0.1, 6)$, on the *same* half. Small κ = strong penalty = very few clauses survive; large κ = weaker penalty = more survive.

4. **Union over the grid.** A clause counts as “selected on this half” if *any* of the 6 fits gave it $|\beta| > \delta$, with $\delta = 10^{-8}$. This means we never have to pick one magic κ — we ask “does this clause survive at *some* sensible sparsity level?”.

Count the votes. For clause j and class ℓ :

$$\hat{\pi}_{\ell j} = \frac{\text{number of the } 2B \text{ halves where clause } j \text{ was selected}}{2B}$$

Keep the stable ones. j enters the signature if $\hat{\pi}_{\ell j} \geq \pi_{\text{thr}}$ (we sweep $\pi_{\text{thr}} \in \{0.5, \dots, 0.9\}$; the multiclass run picks 0.8, i.e. selected in at least 24 of 30 halves).

Read the sign. Average the coefficient $\bar{\beta}_{\ell j}$ over all halves and all 6 grid points (zeros included). Then

$$S_{\ell}^{+} = \{j : \hat{\pi}_{\ell j} \geq \pi_{\text{thr}}, \bar{\beta}_{\ell j} > 0\} \quad (\text{evidence for}), \quad S_{\ell}^{-} = \{\dots, \bar{\beta}_{\ell j} < 0\} \quad (\text{evidence against}).$$

step 1: on each half, fit ℓ_1 logistic at 6 values of κ **step 2: count votes, threshold, read sign**

cell dark = that fit gave the clause a non-zero coefficient

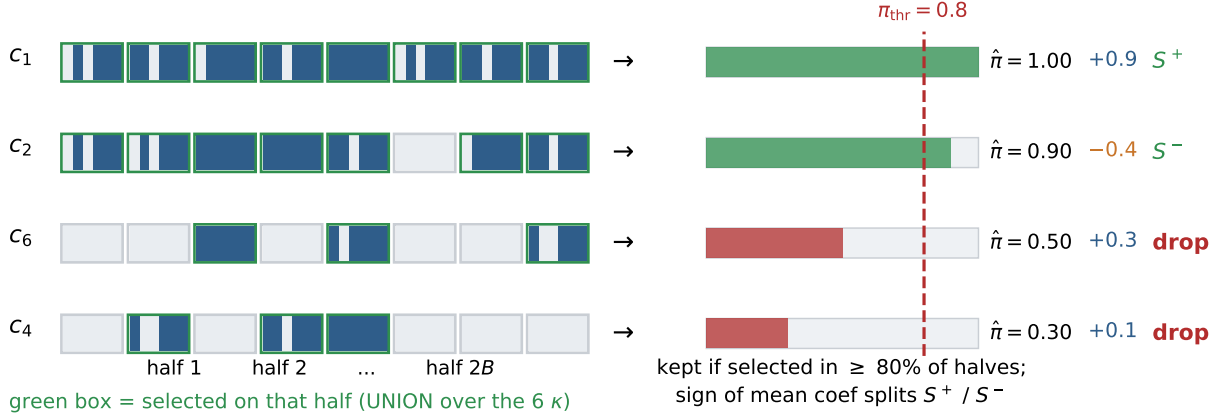


Figure 4: The selection machinery end to end. **Left:** each of the $2B$ halves is fitted at six values of κ ; a clause counts as selected on that half (green box) if *any* of the six fits keeps it. **Right:** the vote fraction $\hat{\pi}_j$ is thresholded, and the sign of the mean coefficient sorts survivors into S^+ and S^- . Numbers match the toy example in Section 4.

4. Worked toy example (tiny numbers, made up for clarity)

Imagine only **6 clauses**, $B = 5$ pairs (so 10 halves), and threshold $\pi_{\text{thr}} = 0.8$. Class = DDoS.

Clause	Selected in	$\hat{\pi}_j$	Mean coef.	Keep?	Meaning
c_1 : high packet rate AND TCP	10/10	1.0	+0.9	YES	supports DDoS
c_2 : many unique dst ports	9/10	0.9	-0.4	YES	speaks against DDoS
c_3 : dst port = 1883 (MQTT)	9/10	0.9	-0.6	YES	speaks against DDoS
c_4 : small packets AND UDP	3/10	0.3	+0.1	no	unstable, drop
c_5 : long flow duration	2/10	0.2	+0.2	no	unstable, drop
c_6 : src port is random	5/10	0.5	+0.3	no	coin-flip, not stable enough

Result: DDoS signature = $S^+ = \{c_1\}$, $S^- = \{c_2, c_3\}$. c_6 was picked half the time — better than nothing, but a clause we cannot reproduce is a clause we cannot testify with, so it is dropped.

5. How a new flow is scored

A new flow x arrives. We check which signature clauses fire on it:

$$s_\ell(x) = \frac{\sum_{j \in S_\ell^+} \hat{\pi}_j z_j(x) - \sum_{j \in S_\ell^-} \hat{\pi}_j z_j(x)}{\sum_{j \in S_\ell^+} \hat{\pi}_j}$$

Each clause votes with its *own reliability* $\hat{\pi}_j$ as the weight, and the denominator normalises so that classes with big signatures are not automatically favoured.

Example (toy numbers above): on flow x , c_1 fires ($z_1=1$), c_2 does not ($z_2=0$), c_3 fires ($z_3=1$):

$$s_{\text{DDoS}}(x) = \frac{(1.0)(1) - (0.9)(0) - (0.9)(1)}{1.0} = \frac{1.0 - 0.9}{1.0} = 0.1$$

We compute this for all 6 families and take the largest. Every point of the score traces back to a printable rule.

6. How do I get the rules back out?

CPSS returns *column indices*, e.g. “clause 436”. Turning an index back into a readable rule is two steps (`scripts/decode_signatures.py`):

1. **Index** $\rightarrow$ **bank**. With $K = 400$ clauses per class, $j = 436$ means source class $\lfloor 436/400 \rfloor = 1$ (DDoS) and clause 36 inside that bank. Clauses in the first half of a bank are *positive-polarity*, the second half *negative-polarity*.
2. **TA states** $\rightarrow$ **literals**. We read `clause.banks[c].get_literals()`. Each clause is a row of 2×154 bits: bit i on means the literal `feature>=bink` is included; bit $154 + i$ on means `NOT(feature>=bink)` is included. The clause is the **AND** of everything included.

Real examples from our run (seed 42, $\pi_{\text{thr}} = 0.8$, DDoS signature):

Clause	$\hat{\pi}_j$	$\bar{\beta}$	Rule
#436, DDoS bank, positive	1.00	+0.451	IF Rate>=bin7 AND NOT(Header.Length>=bin4)
#438, DDoS bank, positive	1.00	+0.128	IF Rate>=bin5 AND syn_flag_number>=bin1 AND rst_flag_number>=bin1
#270, Benign bank, negative	1.00	+0.045	IF Rate>=bin6 (<i>a clause borrowed from another class's bank</i>)

Read clause #436 in plain words: *very high packet rate with a small header* — which is exactly what a flooding attack looks like. That is the kind of sentence we can put in front of a clinician or an auditor.

7. What if clause c_1 has many literals?

Short answer, in two parts.

(a) **CPSS does not care at all.** CPSS only ever sees the column $z_j \in \{0,1\}^n$. Whether that 0/1 came from a 2-literal rule or a 130-literal rule is invisible to the ℓ_1 fit. A long clause is neither penalised nor rewarded by the selection — literal count only matters *afterwards*, when we print the rule for a human.

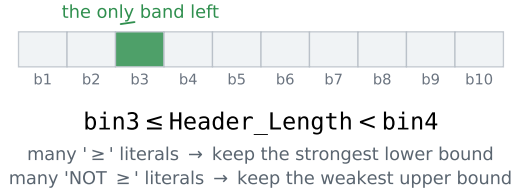
(b) **A long clause is not really long — it collapses.** The thermometer encoding is cumulative ($x \geq \text{bin1}, x \geq \text{bin2}, \dots$), so many literals on the *same* feature merge into one interval:

$$\underbrace{H \geq \text{bin2} \wedge H \geq \text{bin3}}_{\text{keep the strongest: } H \geq \text{bin3}} \wedge \underbrace{\neg H \geq \text{bin4} \wedge \neg H \geq \text{bin5} \wedge \dots}_{\text{keep the weakest: } H < \text{bin4}} \implies \text{bin3} \leq \text{Header_Length} < \text{bin4}$$

raw literals in the clause

Header_Length $\geq$ bin2
Header_Length $\geq$ bin3
NOT(Header_Length $\geq$ bin4)
NOT(Header_Length $\geq$ bin5)
NOT(Header_Length $\geq$ bin6)
... up to bin9

collapse



real clause #1737 in our DDoS signature: 130 literals → 22 feature conditions → 4 informative ones

Figure 5: Why a long clause is not really long: cumulative thermometer bits on the same feature merge into a single interval.

Real case from our run. The longest clause in the seed-42 DDoS signature is #1737 (from the *Recon* bank, $\hat{\pi} = 0.967$, $\bar{\beta} = +0.133$) with **130 literals**. After collapsing, those 130 literals are just **22 feature conditions**, and only four of them carry information:

Collapsed condition	Reading
$\text{bin3} \leq \text{Header_Length} < \text{bin4}$	a narrow header-length band
$\text{bin1} \leq \text{Protocol_Type} < \text{bin2}$	one specific protocol
$\text{Rate} \geq \text{bin2}$	at least moderate packet rate
$\text{bin2} \leq \text{syn_flag_number} < \text{bin3}$	a specific SYN count
ARP, DNS, HTTP, HTTPS, ICMP, SSH, Telnet, SMTP, IRC, IGMP, DHCP, fin/ack/psh/cwr/ece_flag_number, rst_count, rst_flag_number all = off	18 “this protocol/flag is off ” conditions — i.e. “it is none of these other things”

So “130 literals” really reads as: *a moderate-rate flow, one specific protocol, a narrow header band, a specific SYN count, and none of the common application protocols or other TCP flags set*. Long, but still a sentence.

How long are our clauses in practice? Inculpatory signature clauses, seed 42, $\pi_{\text{thr}} = 0.8$:

Class	#clauses	shortest	median	longest
Benign	41	1	5	153
DDoS	29	1	2	130
DoS	33	2	7	119
MQTT	53	1	2	18
Recon	28	1	3	132
Spoofing	50	1	1.5	153

The *typical* signature clause is 1–7 literals. Long clauses exist but they are the tail, not the rule. (The hard ceiling is $2 \times 154 = 308$ literals: 154 thermometer bits plus their negations.)

The one real cost of a long clause. It is very *specific*, so it fires on few flows. That is fine for stability — it can still be selected in all 30 halves if it fires consistently on that family — but it contributes to fewer decisions. Short clauses are the workhorses; long clauses are the sharp corner cases.

8. The exact settings we use

Quantity	Value	Where
Clause pool p	2,400 ($6 \times K$, $K = 400$)	TM: $T = 320$, $s = 8$, 25 epochs
Boolean input	154 bits (22 features $\times$ up to 10 quantile bins)	<code>data_prep.py</code>
CPSS fit rows	40,434	fixed permutation, seed 0
Selection rows	20,217	where (B, π_{thr}) is chosen
Report rows	20,217	never touched until the final number
B grid	$\{5, 10, 15, 20, 25\}$	multiclass picks $B = 15$
π_{thr} grid	$\{0.5, 0.6, 0.7, 0.8, 0.9\}$	multiclass picks 0.8
κ grid Λ	$\text{geomspace}(0.001, 0.1, 6)$	union over the grid
Zero tolerance δ	10^{-8}	$ \beta > \delta$ counts as selected
Logistic fit	liblinear, ℓ_1 , balanced, 200 iters	<code>stabilis.py</code>

The three partitions are disjoint on purpose: CPSS *fits* on one, (B, π_{thr}) is *chosen* on the second, and the reported macro-F1 comes from the third. Choosing the operating point on the same rows we report on would be cheating.

9. Where each step lives

Step	Code	Paper
Build the basis Z	<code>tm.transform(X)</code>	Sec. II-A, Eq. (1)–(2)
Shuffle + split halves	<code>cpss.proportions()</code> , <code>stabilis.py</code>	Sec. II-C
ℓ_1 logistic fit	<code>cpss.proportions()</code>	Sec. II-C
Union over 6 κ	<code>cpss.proportions()</code>	Eq. (6)
Vote count $\hat{\pi}_j$	<code>cpss.proportions()</code>	Eq. (6)
Threshold + sign split	<code>build.signatures()</code>	Eq. (7)
Attribution score	<code>attribution.scores()</code>	Eq. (8)
Error-bound diagnostic	<code>error_bound()</code>	Eq. (9)
Decode index $\rightarrow$ rule	<code>decode.signatures.py</code>	Sec. IV

10. One-paragraph summary

We take the trained TM's 2,400 clauses as our basis: each clause is a readable IF-THEN rule, and each flow becomes a 0/1 vector saying which rules matched. For each attack family we split the fit data into B complementary pairs ($2B$ halves), run a sparse ℓ_1 logistic fit on every half at six regularisation strengths, and count how often each clause is picked. Clauses picked in at least π_{thr} of the halves become the family's signature; the sign of their average coefficient splits them into evidence for (S^+) and evidence against (S^-). A new flow is scored by its signed, reliability-weighted matches. Finally we decode each selected index back into its literal conjunction — typically 1–7 literals, and even a 130-literal clause collapses to about 22 readable feature conditions.